

# Analysis of Differences in PostgreSQL Query Planning for RLS and Views, Including Parallel-Safe Flag Allocation Using logs.

All the Queries: <https://github.com/adityadave29/PE/tree/main/PE-Daywise-Work-Tracker/17-JUNE/Queries>

All the logs (Annotated and highlighted mainly Q4, Q6, and the 3rd query):  
<https://github.com/adityadave29/PE/tree/main/PE-Daywise-Work-Tracker/17-JUNE/Logs>

I have reduced the logs to a total of five.

The **first log** is at the start of the `planner()` function so that we can observe that the view condition is stored inside `subquery->jointree->quals`, whereas the RLS policy condition is stored inside `securityQuals`.

The **second log** is at the start of the `grouping_planner()` function. Before reaching this point, the planner attempts to transform sublinks and subplans into joins. This log will help us see how the planner handles and rewrites view and RLS conditions differently.

The **third log** is at the beginning of plan creation. By this stage, the planner has already explored different paths and selected the cheapest one. This log will show the cheapest path chosen by the planner.

The **fourth log** is at the start of the `ExecutorStart()` function, where we can inspect the execution plan generated and proposed by the planner.

The **fifth log** is inside the `is_parallel_safe()` function. Since we want to understand why certain quals are considered parallel-safe while others are not, I added additional logs in the hazard-checking function called by `is_parallel_safe()`. These logs will display the flags assigned to each qual.

## Query No 4:

Policy:

```
CREATE POLICY orders_rls_policy ON orders
USING (
    EXISTS (
        SELECT 1 FROM customer c
        WHERE c.c_custkey = orders.o_custkey
        AND c.c_acctbal > 5000
    )
);
```

View:

```
CREATE VIEW orders_secure AS
SELECT o.*
FROM orders o
WHERE EXISTS (
    SELECT 1 FROM customer c
    WHERE c.c_custkey = o.o_custkey
    AND c.c_acctbal > 5000
);
```

Query:

```
select
    o_orderpriority,
    count(*) as order_count
from
    orders_secure
where
    o_orderdate >= date '1994-01-01'
    and o_orderdate < date '1994-01-01' + interval '3' month
    and exists (
        select
```

```

        *
    from
        lineitem
    where
        l_orderkey = o_orderkey
        and l_commitdate < l_receiptdate
    )
group by
    o_orderpriority
order by
    o_orderpriority;

```

Log 1: This is the input to the planner.

⇒ The first difference is that the view condition is inside `subquery → jointree → quals`, whereas the RLS policy condition is under `securityQuals`. Other than that, there is no major difference.

Log 2 is after the planner's join operations. Here, there is a significant difference: `securityQuals` are converted into an `ALTERNATIVESUBPLAN`, which means they have not been joined with the user query, while the user query itself has undergone internal join operations.

In the case of views, the planner merges the subquery (the view condition) into the user query, performs multiple join operations, and converts everything into a single query stored under the normal `jointree → quals`.

This is the biggest difference I noticed, and it shows how joins are handled differently in RLS and views.

The second difference is the `parallel_safe` flag. In the RLS plan, it is marked as **false (restricted)**, whereas in the view plan it is marked as **allowed**. As a result, the entire user query can be executed in parallel, but the RLS condition itself cannot be executed in parallel.

This image shows Log 2, where the join operations have already been performed. The `securityQuals` have not been merged with the user query; they remain separate.

```

:securityQuals ((
{ALTERNATIVESUBPLAN
:subplans (
{SUBPLAN
:subLinkType 0
:testexpr <>
:paramIds <>
:plan_id 1
:plan_name SubPlan\ 1
:firstCollType 2278
:firstCollTypmod -1
:firstColCollation 0
:useHashTable false
:unknownEqFalse true
:parallel_safe false
:setParam <>
:parParam (i 0)
:args (
{VAR
:varno 1
:varattno 2
:vartype 23
:vartypmod -1
:varcollid 0
:varnullingrels (b)
:varlevelsup 0
:varreturningtype 0
:varnosyn 1
:varattnosyn 2
:location -1
}
)
:startup_cost 0
:per_call_cost 5922
}
{SUBPLAN
:subLinkType 2
:testexpr
{OPEXPR
:opno 96
:opfuncid 65
:opresulttype 16
:opretset false
:opcollid 0
:inputcollid 0
:args (
{VAR
:varno 1
:varattno 2
:vartype 23
:vartypmod -1
:varcollid 0
:varnullingrels (b)
:varlevelsup 0
:varreturningtype 0
:varnosyn 1
:varattnosyn 2
:location -1
}
{PARAM
:paramkind 1
:paramid 1
:paramtype 23
:paramtypmod -1
:paramcollid 0
:location -1
}
)
:location -1
}
)
)

```

SubPlan 1  
Correlated subplan  
useHashTable false  
Executed per outer row

SubPlan 2  
Hashed subplan  
c\_custkey values hashed once  
useHashTable true

securityQuals are not joined with the user query quals. One interpretation is that the planner has already converted the security quals into a SubPlan, so we no longer see the original predicates here.

This is the parallel\_safe check for the RLS policy predicate. The predicate is not considered parallel-safe because the policy contains a correlated subquery:

```

EXISTS (
  SELECT 1
  FROM customer c
  WHERE c.c_custkey= orders.o_custkey

```

```
)
AND c.c_acctbal>5000
```

For each row of orders, PostgreSQL must first obtain `orders.o_custkey` and then check whether a matching customer exists with `c_acctbal > 5000`. Since the subquery depends on a value from the current outer row (`orders.o_custkey`), PostgreSQL represents it as a correlated `SubPlan`, which is marked as not parallel-safe.

PostgreSQL does not flatten this RLS EXISTS predicate into a Semi Join as aggressively as it does for the equivalent view definition. As a result, the planner retains a correlated SubPlan, which prevents parallel-safe path generation and leads to slower execution.

```
WARNING:  is_parallel_safe: INPUT qual = ((RESTRICTINFO :clause (ALTERNATIVESUBPLAN :subplans ((SUBPLAN :subLinkType 0 :testexpr <> :paramids <> :plan_id 1 :plan_name SubPlan\ 1 :firstColType 2278 :firstColTypeMod -1 :firstColCollation 0 :useHashTable false :unknownEqFalse true :parallel_safe false :setParam <> :parParam (i 0) :args ((VAR :varno 1 :varattno 2 :vartype 23 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 1 :varattnosyn 2 :location -1) :startup_cost 0 :per_call_cost 5922) (SUBPLAN :subLinkType 2 :testexpr (OPEXPR :opno 96 :opfncmd 66 :opresultype 16 :opresult false :opcollid 0 :inputcollid 0 :args ((VAR :varno 1 :varattno 2 :vartype 23 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 1 :varattnosyn 2 :location -1) (PARAM :paramkind 1 :paramid 1 :paramtype 23 :paratypmod -1 :paramcollid 0 :location -1) :location -1) :paramids (i 1) :plan_id 2 :plan_name SubPlan\ 2 :firstColType 23 :firstColTypeMod -1 :firstColCollation 0 :useHashTable true :unknownEqFalse true :parallel_safe true :setParam <> :parParam <> :args <> :startup_cost 5716.52 :per_call_cost 0.0025))) :is_pushed_down true :can_join false :pseudoconstant false :has_clone false :leakproof false :has_volatile 0 :security_level 0 :num_base_rels 1 :clause_rels (b 1) :required_rels (b 1) :incompatible_rels (b) :outer_rels (b) :left_rels (b) :orclause <> :rinfo_serial 1 :eval_cost.startup -1 :eval_cost.per_tuple 0 :norm_selec -1 :outer_selec -1 :mergeofamilies <> :left_em <> :right_em <> :outer_is_left false :hashjoinoperator 0 :left_bucketsize -1 :right_bucketsize -1 :left_mcvfreq -1 :right_mcvfreq -1 :left_hashoperator 0 :right_hashoperator 0) (RESTRICTINFO :clause (OPEXPR :opno 1098 :opfncmd 1090 :opresultype 16 :opresult false :opcollid 0 :inputcollid 0 :args ((VAR :varno 1 :varattno 5 :vartype 1082 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 1 :varattnosyn 5 :location -1) (CONST :consttype 1082 :consttypmod -1 :constcollid 0 :constlen 4 :constbval true :constisnull false :location -1 :constvalue 4 [ 113 -9 -1 -1 -1 -1 -1 ])) :location -1) :is_pushed_down true :can_join false :pseudoconstant false :has_clone false :is_clone false :leakproof false :has_volatile 0 :security_level 1 :num_base_rels 1 :clause_rels (b 1) :required_rels (b 1) :incompatible_rels (b) :outer_rels (b) :left_rels (b) :orclause <> :rinfo_serial 3 :eval_cost.startup -1 :eval_cost.per_tuple 0 :norm_selec -1 :outer_selec -1 :mergeofamilies <> :left_em <> :right_em <> :outer_is_left false :hashjoinoperator 0 :left_bucketsize -1 :right_bucketsize -1 :left_mcvfreq -1 :right_mcvfreq -1 :left_hashoperator 0 :right_hashoperator 0))
WARNING:  max_parallel_hazard_walker: SubPlan plan_id=1 parallel_safe=0 > reason=correlated_subplan
WARNING:  is_parallel_safe: OUTPUT qual = ((RESTRICTINFO :clause (ALTERNATIVESUBPLAN :subplans ((SUBPLAN :subLinkType 0 :testexpr <> :paramids <> :plan_id 1 :plan_name SubPlan\ 1 :firstColType 2278 :firstColTypeMod -1 :firstColCollation 0 :useHashTable false :unknownEqFalse true :parallel_safe false :setParam <> :parParam (i 0) :args ((VAR :varno 1 :varattno 2 :vartype 23 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 1 :varattnosyn 2 :location -1) :startup_cost 0 :per_call_cost 5922) (SUBPLAN :subLinkType 2 :testexpr (OPEXPR :opno 96 :opfncmd 66 :opresultype 16 :opresult false :opcollid 0 :inputcollid 0 :args ((VAR :varno 1 :varattno 2 :vartype 23 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 1 :varattnosyn 2 :location -1) (PARAM :paramkind 1 :paramid 1 :paramtype 23 :paratypmod -1 :paramcollid 0 :location -1) :location -1) :paramids (i 1) :plan_id 2 :plan_name SubPlan\ 2 :firstColType 23 :firstColTypeMod -1 :firstColCollation 0 :useHashTable true :unknownEqFalse true :parallel_safe true :setParam <> :parParam <> :args <> :startup_cost 5716.52 :per_call_cost 0.0025))) :is_pushed_down true :can_join false :pseudoconstant false :has_clone false :is_clone false :leakproof false :has_volatile 0 :security_level 0 :num_base_rels 1 :clause_rels (b 1) :required_rels (b 1) :incompatible_rels (b) :outer_rels (b) :left_rels (b) :orclause <> :rinfo_serial 1 :eval_cost.startup -1 :eval_cost.per_tuple 0 :norm_selec -1 :outer_selec -1 :mergeofamilies <> :left_em <> :right_em <> :outer_is_left false :hashjoinoperator 0 :left_bucketsize -1 :right_bucketsize -1 :left_mcvfreq -1 :right_mcvfreq -1 :left_hashoperator 0 :right_hashoperator 0) (RESTRICTINFO :clause (OPEXPR :opno 1098 :opfncmd 1090 :opresultype 16 :opresult false :opcollid 0 :inputcollid 0 :args ((VAR :varno 1 :varattno 5 :vartype 1082 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 1 :varattnosyn 5 :location -1) (CONST :consttype 1082 :consttypmod -1 :constcollid 0 :constlen 4 :constbval true :constisnull false :location -1 :constvalue 4 [ 113 -9 -1 -1 -1 -1 -1 ])) :location -1) :is_pushed_down true :can_join false :pseudoconstant false :has_clone false :is_clone false :leakproof true :has_volatile 0 :security_level 1 :num_base_rels 1 :clause_rels (b 1) :required_rels (b 1) :incompatible_rels (b) :outer_rels (b) :left_rels (b) :orclause <> :rinfo_serial 2 :eval_cost.startup -1 :eval_cost.per_tuple 0 :norm_selec -1 :outer_selec -1 :mergeofamilies <> :left_em <> :right_em <> :outer_is_left false :hashjoinoperator 0 :left_bucketsize -1 :right_bucketsize -1 :left_mcvfreq -1 :right_mcvfreq -1 :left_hashoperator 0 :right_hashoperator 0) (RESTRICTINFO :clause (OPEXPR :opno 1098 :opfncmd 1090 :opresultype 16 :opresult false :opcollid 0 :inputcollid 0 :args ((VAR :varno 1 :varattno 5 :vartype 1082 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 1 :varattnosyn 5 :location -1) (CONST :consttype 1082 :consttypmod -1 :constcollid 0 :constlen 4 :constbval true :constisnull false :location -1 :constvalue 4 [ 113 -9 -1 -1 -1 -1 -1 ])) :location -1) :is_pushed_down true :can_join false :pseudoconstant false :has_clone false :is_clone false :leakproof false :has_volatile 0 :security_level 1 :num_base_rels 1 :clause_rels (b 1) :required_rels (b 1) :incompatible_rels (b) :outer_rels (b) :left_rels (b) :orclause <> :rinfo_serial 3 :eval_cost.startup -1 :eval_cost.per_tuple 0 :norm_selec -1 :outer_selec -1 :mergeofamilies <> :left_em <> :right_em <> :outer_is_left false :hashjoinoperator 0 :left_bucketsize -1 :right_bucketsize -1 :left_mcvfreq -1 :right_mcvfreq -1 :left_hashoperator 0 :right_hashoperator 0))
parallel_safe = false | reason = PROPPARALLEL_RESTRICTED
```

## Query 6:

Policy:

```
CREATE POLICY lineitem_rls_policy ON lineitem
USING (
    EXISTS (
        SELECT 1 FROM supplier s
        WHERE s.s_suppkey = lineitem.l_suppkey
        AND s.s_acctbal > 1000
```

```
)  
);
```

View:

```
CREATE VIEW lineitem_secure AS  
SELECT l.*  
FROM lineitem l  
WHERE EXISTS (  
    SELECT 1 FROM supplier s  
    WHERE s.s_suppkey = l.l_suppkey  
    AND s.s_acctbal > 1000  
);
```

Query:

```
select  
    sum(l_extendedprice * l_discount) as revenue  
from  
    lineitem_secure  
where  
    l_shipdate >= date '1993-01-01'  
and l_shipdate < date '1994-03-01' + interval '1' year  
and l_discount between 0.06 - 0.01 and 0.06 + 0.01  
and l_quantity < 10;
```

In Q6, the same thing happened as in Q4. The `securityQuals` were kept in a separate plan, whereas in the view case, the planner performed join operations and merged the view condition into the main query.

View:

```

)
:jointree
{FROMEXPR
:fromlist (
{JOINEXPR
:jointype 4
:isNatural false
:larg
{FROMEXPR
:fromlist (
{RANGETBLREF
:rtindex 2
}
)
:quals <>
}
:rarg
{RANGETBLREF
:rtindex 3
}
}

```

lineitem SEMI JOIN supplier

RLS After pull up:

```

:securityQuals ((
{ALTERNATIVESUBPLAN
:subplans (
{SUBPLAN
:subLinkType 0
:testexpr <>
:paramIds <>
:plan_id 1
:plan_name SubPlan\ 1
:firstColType 2278
:firstColTypmod -1
:firstColCollation 0
:useHashTable false
:unknownEqFalse true
:parallel_safe false
:setParam <>
:parParam (i 0)
:args (
{VAR
:varno 1
:varattno 3
:vartype 23
:vartypmod -1
:varcollid 0
:varnullingrels (b)
:varlevelsup 0
:varreturningtype 0
:varnosyn 1
:varattnosyn 3
:location -1
}
)
:startup_cost 0
:per_call_cost 368
}
{SUBPLAN
:subLinkType 2
:testexpr
{OPEXPR
:opno 96
:opfuncid 65
:opresulttype 16
:opretset false
:opcollid 0
:inputcollid 0
}
}
)
)

```

So now I need to try a policy whose conditions are not related to the user query and then check whether the planner marks the policy as `parallel_safe` or not. The next query will not be from the TPC-H Q1-Q22 benchmark queries.

## 3rd Query:

Policy:

```
CREATE POLICY lineitem_rls_policy ON lineitem
USING (
    EXISTS (
        SELECT 1
        FROM supplier s
        WHERE s.s_acctbal > 1000
    )
);
```

Views:

```
CREATE VIEW lineitem_secure AS
SELECT l.*
FROM lineitem l
WHERE EXISTS (
    SELECT 1
    FROM supplier s
    WHERE s.s_acctbal > 1000
);
```

Query:

```
SELECT count(*)
FROM lineitem_secure;
```

RLS:

```

-----
Finalize Aggregate (cost=133849.57..133849.58 rows=1 width=8) (actual time=639.519..643.710 rows=1.00 loops=1)
  Output: count(*)
  Buffers: shared hit=2258 read=102464
  InitPlan 1
    -> Seq Scan on public.supplier s (cost=0.00..343.00 rows=8196 width=0) (actual time=0.058..0.058 rows=1.00 loops=1)
      Filter: (s.s_acctbal > '1000')::double precision)
      Buffers: shared hit=2
    -> Gather (cost=133849.31..133849.52 rows=2 width=8) (actual time=639.383..643.705 rows=3.00 loops=1)
      Output: (PARTIAL count(*))
      Workers Planned: 2
      Workers Launched: 2
      Buffers: shared hit=2258 read=102464
      -> Partial Aggregate (cost=132849.31..132849.32 rows=1 width=8) (actual time=624.820..624.820 rows=1.00 loops=3)
        Output: PARTIAL count(*)
        Buffers: shared hit=2256 read=102464
        Worker 0: actual time=615.019..615.019 rows=1.00 loops=1
          Buffers: shared hit=717 read=33620
        Worker 1: actual time=620.442..620.442 rows=1.00 loops=1
          Buffers: shared hit=747 read=34248
      -> Parallel Seq Scan on public.lineitem (cost=0.00..129723.83 rows=1250192 width=0) (actual time=1.215..536.321 rows=2000405.00 loops=3)
        Output: lineitem.l_orderkey, lineitem.l_partkey, lineitem.l_suppkey, lineitem.l_linenum, lineitem.l_quantity, lineitem.l_extendedprice, lineitem.l_discount, lineitem.l_tax, lineitem.l_returnflag, lineitem.l_linestatus, lineitem.l_shipdate, lineitem.l_commitdate, lineitem.l_receiptdate, lineitem.l_shipinstruct, lineitem.l_shipmode, lineitem.l_comment
        Filter: ((initplan 1).col1)
        Buffers: shared hit=2256 read=102464
        Worker 0: actual time=1.254..528.583 rows=1967897.00 loops=1
          Buffers: shared hit=717 read=33620
        Worker 1: actual time=1.506..532.148 rows=2005613.00 loops=1
          Buffers: shared hit=747 read=34248
  Planning:
    Buffers: shared hit=21
  Planning Time: 12.151 ms
  Execution Time: 647.307 ms
(31 rows)

```

## Views:

```

-----
QUERY PLAN

Finalize Aggregate (cost=136975.05..136975.06 rows=1 width=8) (actual time=647.525..652.113 rows=1.00 loops=1)
  Output: count(*)
  Buffers: shared hit=2540 read=102182
  InitPlan 1
    -> Seq Scan on public.supplier s (cost=0.00..343.00 rows=8196 width=0) (actual time=0.333..0.333 rows=1.00 loops=1)
      Filter: (s.s_acctbal > '1000')::double precision)
      Buffers: shared hit=2
    -> Gather (cost=136974.79..136975.00 rows=2 width=8) (actual time=647.392..652.104 rows=3.00 loops=1)
      Output: (PARTIAL count(*))
      Workers Planned: 2
      Workers Launched: 2
      Buffers: shared hit=2540 read=102182
      -> Partial Aggregate (cost=135974.79..135974.80 rows=1 width=8) (actual time=630.607..630.608 rows=1.00 loops=3)
        Output: PARTIAL count(*)
        Buffers: shared hit=2538 read=102182
        Worker 0: actual time=624.043..624.043 rows=1.00 loops=1
          Buffers: shared hit=798 read=33782
        Worker 1: actual time=621.700..621.709 rows=1.00 loops=1
          Buffers: shared hit=856 read=33456
      -> Result (cost=0.00..129723.83 rows=2500383 width=0) (actual time=0.678..537.348 rows=2000405.00 loops=3)
        One-Time Filter: ((initplan 1).col1)
        Buffers: shared hit=2538 read=102182
        Worker 0: actual time=0.932..531.722 rows=1981504.00 loops=1
          Buffers: shared hit=798 read=33782
        Worker 1: actual time=0.811..529.832 rows=1966212.00 loops=1
          Buffers: shared hit=856 read=33456
      -> Parallel Seq Scan on public.lineitem l (cost=0.00..129723.83 rows=2500383 width=0) (actual time=0.677..374.821 rows=2000405.00 loops=3)
        Output: l.l_orderkey, l.l_partkey, l.l_suppkey, l.l_linenum, l.l_quantity, l.l_extendedprice, l.l_discount, l.l_tax, l.l_returnflag, l.l_linestatus, l.l_shipdate, l.l_commitdate, l.l_receiptdate, l.l_shipinstruct, l.l_shipmode, l.l_comment
        Buffers: shared hit=2538 read=102182
        Worker 0: actual time=0.931..371.287 rows=1981504.00 loops=1
          Buffers: shared hit=798 read=33782
        Worker 1: actual time=0.810..370.959 rows=1966212.00 loops=1
          Buffers: shared hit=856 read=33456
  Planning:
    Buffers: shared hit=9
  Planning Time: 12.607 ms
  Execution Time: 654.326 ms
(37 rows)

```

Here we can see that the execution times for RLS and views are the same. The reason is that the 3rd query is very simple, and the policy condition does not need to be evaluated for each row of the user query, unlike in Q4 and Q6. As a result, both the RLS and view conditions can be executed in parallel. This is also why, in the RLS logs after pull-up, the original `securityQuals` are no longer visible—they have been transformed into a precomputed parameter and are no longer restricted by the user query. These three queries help explain why RLS can be slower than views in cases such as Q4 and Q6.

Now I am performing another experiment in which I will execute the same 3rd query using a UDF. In one version, the function will be marked as `PARALLEL SAFE`, and in the other version, I will not specify any parallel-safety option. From the

PostgreSQL documentation, I learned that a user-defined function is treated as parallel unsafe by default unless it is explicitly marked as `PARALLEL SAFE`. Therefore, PostgreSQL may prevent parallel execution when the function does not have the appropriate parallel-safety flag.

## UDF EXPERIMENT with PARALLEL SAFE

UNSAFE:

```
Aggregate (cost=172231.20..172231.21 rows=1 width=8) (actual time=987.437..987.438 rows=1.00 loops=1)
  Output: count(*)
  Buffers: shared hit=3030 read=101900
  InitPlan 1
    -> Seq Scan on public.supplier s (cost=0.00..2818.00 rows=3333 width=0) (actual time=2.584..2.585 rows=1.00 loops=1)
        Filter: check_supplier_balance_unsafe(s.s_acctbal)
        Buffers: shared hit=210
    -> Seq Scan on public.lineitem (cost=0.00..164729.20 rows=3000400 width=0) (actual time=3.683..696.180 rows=6001215.00 loops=1)
        Output: lineitem.l_orderkey, lineitem.l_partkey, lineitem.l_suppkey, lineitem.l_linenum, lineitem.l_quantity, lineitem.l_extendedprice, lineitem.l_discount, lineitem.l_tax, lineitem.l_returnflag, lineitem.l_linestatus, lineitem.l_shipdate, lineitem.l_commitdate, lineitem.l_receiptdate, lineitem.l_shipinstruct, lineitem.l_shipmode, lineitem.l_comment
        Filter: (initPlan 1).col1
        Buffers: shared hit=3030 read=101900
  Planning:
    Buffers: shared hit=24
  Planning Time: 3.589 ms
  Execution Time: 992.268 ms
(15 rows)
```

SAFE:

```
Finalize Aggregate (cost=133850.37..133850.38 rows=1 width=8) (actual time=627.577..628.695 rows=1.00 loops=1)
  Output: count(*)
  Buffers: shared hit=3132 read=101806
  InitPlan 1
    -> Seq Scan on public.supplier s (cost=0.00..2818.00 rows=3333 width=0) (actual time=2.209..2.209 rows=1.00 loops=1)
        Filter: check_supplier_balance_safe(s.s_acctbal)
        Buffers: shared hit=218
    -> Gather (cost=133849.31..133849.52 rows=2 width=8) (actual time=627.398..628.686 rows=3.00 loops=1)
        Output: (PARTIAL count(*))
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=3132 read=101806
    -> Partial Aggregate (cost=132849.31..132849.32 rows=1 width=8) (actual time=612.479..612.480 rows=1.00 loops=3)
        Output: PARTIAL count(*)
        Buffers: shared hit=2914 read=101806
        Worker 0: actual time=606.116..606.117 rows=1.00 loops=1
        Buffers: shared hit=974 read=33536
        Worker 1: actual time=606.385..606.385 rows=1.00 loops=1
        Buffers: shared hit=959 read=33696
    -> Parallel Seq Scan on public.lineitem (cost=0.00..129723.83 rows=1250192 width=0) (actual time=0.692..529.988 rows=2000405.00 loops=3)
        Output: lineitem.l_orderkey, lineitem.l_partkey, lineitem.l_suppkey, lineitem.l_linenum, lineitem.l_quantity, lineitem.l_extendedprice, lineitem.l_discount, lineitem.l_tax, lineitem.l_returnflag, lineitem.l_linestatus, lineitem.l_shipdate, lineitem.l_commitdate, lineitem.l_receiptdate, lineitem.l_shipinstruct, lineitem.l_shipmode, lineitem.l_comment
        Filter: (initPlan 1).col1
        Buffers: shared hit=2914 read=101806
        Worker 0: actual time=0.338..525.557 rows=1977814.00 loops=1
        Buffers: shared hit=974 read=33536
        Worker 1: actual time=0.445..524.799 rows=1985993.00 loops=1
        Buffers: shared hit=959 read=33696
  Planning:
    Buffers: shared hit=27
  Planning Time: 5.421 ms
  Execution Time: 637.458 ms
(31 rows)
```

Did the experiment: the query with the `PARALLEL SAFE` function and policy took **637 ms**, while the same query without the `PARALLEL SAFE` flag took **992 ms**. This demonstrates the importance of the parallel-safety flag and its impact on query performance.

In the planner, we can broadly divide the process into three parts. The first part is from `planner()` to `grouping_planner()`, where the planner performs query transformations and all valid join operations. The second part involves exploring all possible execution paths, and the third part creates the final plan using the

cheapest path found in the previous stage. **The findings discussed above were derived from the first part of the planner.**